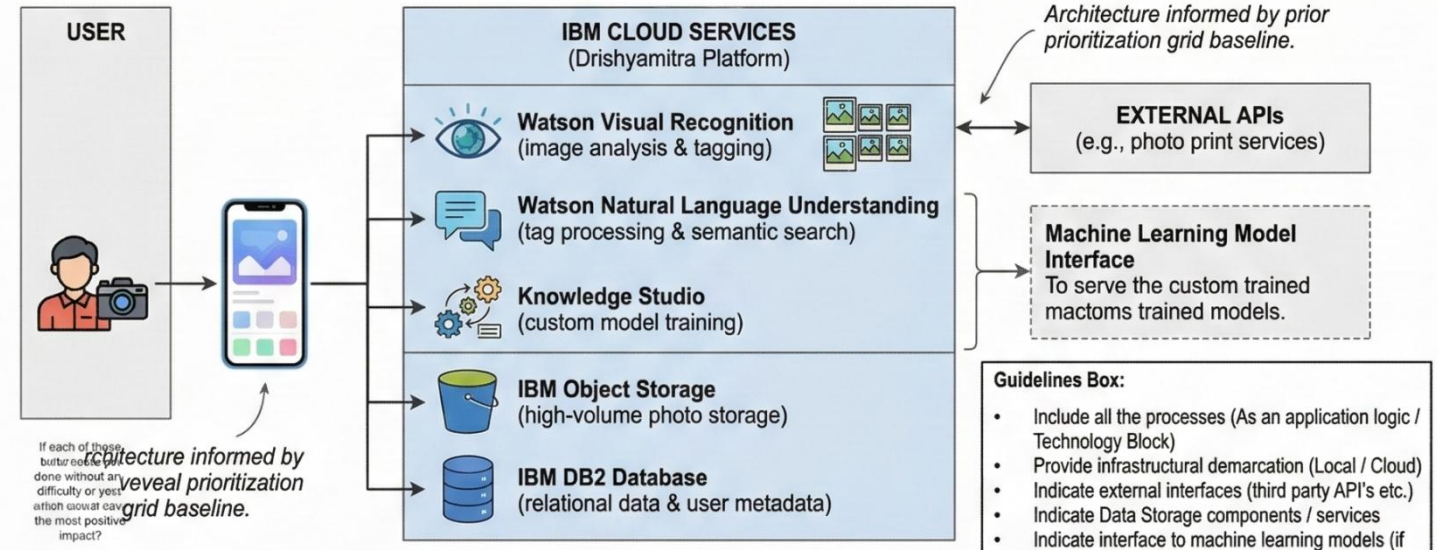
Project Design Phase-II
Technology Stack (Architecture & Stack)

Date	31 January 2026
Team ID	SWUID202642146
Project Name	Drishyamitra AI-Powered Photo Management System
Maximum Marks	4 Marks



Guidelines Box:

- Include all the processes (As an application logic / Technology Block)
- Provide infrastructural demarcation (Local / Cloud)
- Indicate external interfaces (third party API's etc.)
- Indicate Data Storage components / services
- Indicate interface to machine learning models (if applicable)

Technology Stack Table

S.No.	Component	Description	Technology
1.	User Interface	How user interacts with application e.g. Web UI, Mobile App, Chatbot etc.	React.js / Swift / Kotlin
2.	Application Logic	Back-end services with backend application management syeces.	Python with FastAPI / Docker
3.	Watson Services	AI Model integration with nodel and red photo analyslis for photos.	Watson Visual Recognition, NLU, custom models via Knowledge Studio
4.	Data Storage	Core database and photo binaries	PostgreSQL & IBM Object Storage
5.	External APIs	Print service & sharing interfaces	Photo Print Service API, Social Media APIs
6.	Machine Learning ML Interface	Custom model serving and deployment	REST APIs for custom visual model endpoints

S.No	Characteristics	Description	Technology
1.	Open-Source Frameworks	Django (Python) is used for the backend to build robust APIs and handle AI model integration. Keras with TensorFlow is used for developing and deploying AI models for image recognition and auto-tagging. Angular (TypeScript) provides a component-based frontend for a responsive and intuitive user interface.	**Backend:** Django Framework (v4.2) **AI/ML:** Keras & TensorFlow (v2.12) **Frontend:** Angular Framework (v16.1.6)
2.	Security Implementations	A comprehensive security layer is implemented, including HTTPS/TLS 1.3 for data encryption in transit and AES-256 for data at rest. JWT (JSON Web Tokens) are used for secure user authentication and authorization. Input validation and adherence to OWASP guidelines prevent common application vulnerabilities. IAM policies restrict access to backend resources and databases.	**Encryption:** TLS 1.3, AES-256 **Authentication:** JWT (JSON Web Tokens) **Application Security:** AWS IAM, OWASP ZAP (for regular security scanning)
3.	Scalable Architecture	The system adopts a microservices architecture. Bulk photo processing and AI inference are handled by serverless functions that automatically scale based on workload. Images are stored and retrieved through a content delivery network to reduce latency. The database uses a distributed NoSQL model to accommodate growing metadata without sacrificing performance.	**Compute:** AWS Lambda, Amazon EKS **Messaging:** Amazon SQS **CDN:** Amazon CloudFront **Database:** Amazon DynamoDB
4.	Availability	High availability is achieved by deploying application and database servers across multiple geographic regions and zones. A reliable DNS and traffic management service reroutes user requests in case of a service outage in one region. Automatic database backups and point-in-time recovery ensure minimal data loss.	**Compute:** AWS Application Load Balancer **Database:** Amazon RDS (Multi-AZ Deployment) **Storage:** Amazon S3 **Traffic Management:** AWS Route 53

Table-2: Application Characteristics:

S.No	Characteristics	Description	Technology
1.	**Open-Source Frameworks**	Utilizing Python for its vast ML ecosystem (PyTorch for deep learning models), with a Django backend for robust API development and a React frontend for an intuitive user experience.	Python 3.10+, PyTorch 1.12+, Django 4.2+, React 18+
2.	**Security Implementations**	A multi-layered approach: SHA-256 for password hashing, OAuth 2.0 via Django Allauth for user authentication, AWS IAM for infrastructure control, and Amazon WAF to mitigate common OWASP top 10 exploits.	JWT, SHA-256, HTTPS (TLS 1.3), OAuth 2.0, AWS WAF, AWS IAM, Snyk code scanning
3.	**Scalable Architecture**	A containerized microservices architecture managed by Kubernetes (EKS). Compute-intensive AI inferencing (object detection, classification) scales independently, decoupled from user services by a task queue.	Kubernetes (EKS), Docker, AWS Lambda (for certain photo conversions), Celery with RabbitMQ, Amazon RDS (PostgreSQL), Amazon Elasticsearch Service
4.	**Availability**	Achieving >99.95% uptime through multi-Availability Zone (AZ) deployments of EKS clusters, multi-AZ RDS databases, and a global Amazon CloudFront CDN. AWS Route 53 manages health checks.	AWS ALB (Application Load Balancer), Multi-AZ RDS, Amazon S3 Cross-Region Replication, Route 53, Prometheus & Grafana
5.	**Performance**	Design target >150 requests per second. Redis caching for frequently accessed user profiles and search queries. Amazon CloudFront CDN for content delivery to minimize photo loading latency. Optimized ONNX-runtime for fast AI inference.	Amazon CloudFront, Redis, ONNX Runtime, NGINX as reverse proxy